Copilot 需求交付 Skill 如何实现数据需求24h交付

原创 直播技术数据 大淘宝技术 2026年7月19日 23:44 浙江 9人



本文介绍了一个面向数据开发团队的数据需求交付 Agent Skill。该技能通过自然语言交互（可以是从 Aone、钉钉消息、PRD 中提取的描述），输出是一份结构化的 Copilot 交互物(prompt)，可以直接复制到 DataWorks Copilot 或调用其他大模型中生成高质量 SQL 代码。让数据研发从“依赖人的连续投入的串行排期”转变为“Agent自动执行，人按优先级介入决策的并行推进”。文章从背景痛点、skill介绍、实战场景、核心能力拆解等方面系统展开，并在最后给出总结与展望，希望为同样在做数据质量保障和 Agent 工具落地的开发者提供参考。（文章内容基于作者个人技术实践与独立思考，旨在分享经验，仅代表个人观点。）

01 前置说明

技术栈与适用范围

本方案基于以下技术栈实现，读者可以参考其中的设计思路将类似方案适配到自己的环境：

- 计算引擎：MaxCompute（ODPS），云端大数据计算服务
- 数据开发平台：阿里云 DataWorks，提供表结构查询、节点代码获取、数据血缘追溯、SQL 执行等 API 能力
- Agent 运行时：支持 Skill 定义和自然语言交互的 Agent 框架

核心设计思路（标准化模板、基准表发现策略、降级验数策略等），适用于任何有标准数据建模、元数据查询 API （表结构、表代码）和 SQL 执行能力的数仓研发平台。

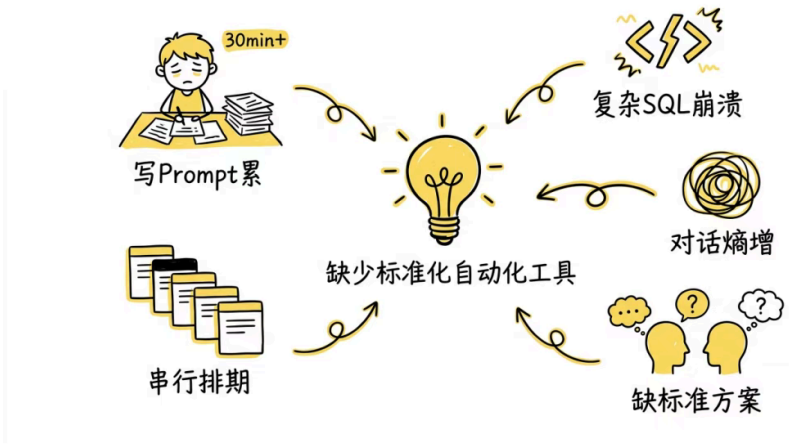
术语解释

术语	含义
ADS	Application Data Store，应用数据层，面向业务场景的宽表/CUBE 表
DWS	Data Warehouse Summary，汇总数据层，按主题域轻度聚合
DWD	Data Warehouse Detail，明细数据层，清洗后的事实明细表
DIM	Dimension，维度表，描述业务实体属性的参照表
DataWorks Copilot	大数据开发治理平台 DataWorks 平台提供的 AI Agent，能够根据自然语言完成多种SQL操作，包括SQL生成，SQL改写，SQL纠错及生成注释。

02 背景与痛点

在淘宝直播数据团队内，大量同学已经习惯了通过 DataWorks Copilot 等生码工具生成 SQL，提升需求交付的效率 。现阶段SOTA的代码模型生码质量确实强大——一个简单的取数需求，如果口径清晰、来源表明确、数据模型定义准确（表与表关联关系、业务语义翻译），通过和 Copilot 对话，大约十分钟以内就能得到一段主体框架正确，基本能运行的代码。

但问题在于：**"口径清晰、来源表明确、数据模型定义准确"这个前提，在真实的工作场景里恰恰是不具备的**。用了半年后，笔者总结出了当前在使用AI生成SQL代码时的几个痛点。



痛点 1：写 Prompt 这件事本身就很累

大家都知道 Copilot 生码质量取决于 Prompt 质量，但问题是：

你要自己从需求描述里提取关键信息，自己回忆或翻文档找来源表，自己梳理字段映射关系和加工口径，自己组织一段结构化的提示词。这整个过程全靠人脑跑，一次复杂需求光"写 Prompt 准备工作"就要半小时起步。那可能还不如自己手写代码了。

本质问题：Prompt Engineering 是一套独立的心智负担。编写时用户脑子里同时要装业务口径 + 表结构 + 团队代码规范 + Copilot 的偏好格式，本身就要花费时间。

痛点 2：复杂业务逻辑生码质量差

一旦遇到业务语义模糊、逻辑复杂需求，Copilot 基本就"崩了"：

> 示例：「xx应用事件埋点看板：从商品 id × 商品版本 × 主播 id，计算全链路 12 个事件的耗时、重试次数」

这个需求有多个数据源、多层 CTE 嵌套、复杂的配对逻辑（比如"取跟着 事件id为 1019 后的第一条 1021/1023/1026"），直接丢给 Copilot 的结果通常是：

- 编造不存在的表名
- 不遵守离线研发规范或指标命名规范
- 数据模型设计混乱，业务语义不能准确翻译成 SQL 代码逻辑

造成的结果就是，把需求 PRD 简单处理后输入给大模型生成代码，改 bug 的时间比从头开始写还多。

本质问题：LLM 对"复杂嵌套逻辑"和"隐含业务约束"的理解能力有限，复杂业务逻辑需求的模型设计、需求拆解还是依赖有经验的数据研发同学来完成。

更深入地分析：当前网页端 Copilot 的交互模式没有 spec 约束，大模型当然可以"猜"，但当它在业务逻辑翻译、表与表关联、聚合粒度上猜错其中一步时，整段 SQL 就废了——而且用户很难指导模型修改，只能整段重写。

解决的方法就是 **Spec Coding（规约驱动编程）** ——在让 AI 写代码之前，先让 AI 帮你把"要写什么"搞明白，产出一份双方确认过的规格说明书（Spec），再基于 Spec 让 AI 写代码。代码生成不再是"黑盒魔法"，而是"翻译"——把经过人类 review 的模型设计翻译成代码。

痛点 3：多轮对话的"熵增定律"

Copilot Agent 模式可以多轮对话，上一轮的结果可以作为下一轮的上下文。这本来是好事——但实际体验是：

你在一个session的对话里完成了需求输入、问题澄清后，模型生成了代码，但是你还是发现有几处地方有问题。当你通过和它对话修复了部分错误以后，上下文越来越长，模型思考越来越久，效果也有一定衰减。这个时候，你想开一个新对话——那就得把之前整理好的需求内容、表结构、纠错的对话**从头再跟它说一遍**。

本质问题：多轮对话的上下文越来越长，网页端生码不适合长程沟通，需要拆解需求完成过程，对中间过程状态做落盘归档。

痛点 4：需求评审后没有明确的产物

这个痛点藏得最深，但杀伤力最大。

需求评审以后没有明确的产物，对需求的理解停留在需求描述、钉钉聊天记录和大家的脑子里。例如：业务方说"确收成交金额"，数据同学认为就是普通口径的确收成交金额，但是没想到业务真实要的是锁支付确收成交金额。几天后需求交付，验数的过程中，业务方说"这个数不对"，数据同学查了才发现——口径理解不一致。

没有一份**双方都认可的结构化需求方案**做锚点，所有口头共识都是薛定谔的共识——在真正看数据之前，你永远不知道它是"一致"还是"不一致"。

本质问题：缺少一份标准化的、需求方和数据方共同对齐的中间产物。PRD 太粗，SQL 太细，中间缺了一层"数据语义翻译层"。

痛点 5：多需求只能串行排期，交付效率低

当数据研发同学同时承接 3–5 个需求时，每个需求都需要经历：整理需求文档 → 查询指标口径和相关表 → 制定技术方案 → 编码 → 验数。这些步骤中，"整理文档、查口径、定方案"占据了大量时间，而且高度依赖人的连续注意力——你不可能在写 A 需求的技术方案时，同时去查 B 需求的指标口径。

结果就是：**需求只能串行完成，后面的需求只能排队等**。即使某些需求本身复杂度不高，也因为"前置准备工作"的时间成本被迫延后。

本质问题：需求交付的前置准备工作（需求澄清、口径确认、表结构梳理、技术方案制定）耗时长且依赖人的连续投入，无法并行化。如果这些工作能被 Agent 分担，数据研发同学就可以同时推进多个需求的"准备阶段"，只在关键决策点（需求澄清 确认、模型设计 CR）介入，从而实现**多需求并行推进**。

痛点总结

五个痛点串起来看，其实指向同一个根因：

> **从“业务需求”到“高质量 Prompt”这一段，缺少一个标准化、可复用、不依赖个人经验的自动化工具，帮助数据研发同学提升效率。**

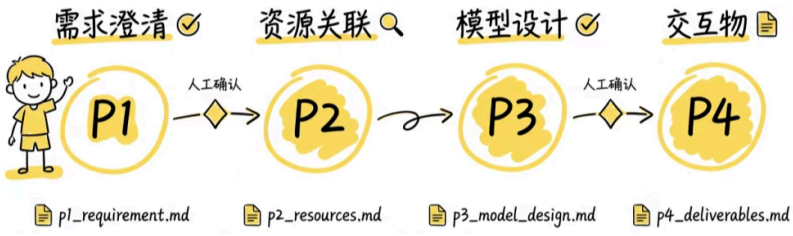
因此，笔者做了：`Copilot 需求交付 Skill（copilot_req2sql）`。整个Skill分为4个阶段工作流，产出的 P1–P4 文档，本质上就是一份**分阶段确认的 Spec 文档链**：从需求澄清 → 资源梳理 → 模型设计 → 交互物 Prompt，每一步都是一个不可篡改的"锚点"，上一步不确认就不推进下一步。

03 copilot_req2sql 介绍

一句话定义

`copilot_req2sql` 是一个**数据需求交付 Agent Skill**。它的输入是一份业务需求（可以是从 Aone、钉钉消息、PRD 中提取的描述），输出是一份结构化的 **Copilot 交互物（prompt）**，可以直接复制到 DataWorks Copilot 或其他大模型中生成高质量 SQL 代码。

整个过程通过和Agent对话完成，包含 4 个阶段：需求澄清、资源管理、模型设计、交付物产出。



Spec 目录结构

Skill 执行过程中会自动创建规范的目录结构，每个需求对应一个独立的 Spec 目录：

```
specs/
├─ yyyymmdd_{任务名}/
    ├── stages/                # agent工作流程
        ├── p1_requirement.md              # 需求澄清结果
        ├── p2_resources.md               # 相关表和伪代码
        ├── p3_model_design.md           # 模型设计
        ├── p4_copilot_input.md           # 给copilot的input
        ├── workflow_events.jsonl            # 工作流事件记录
    ├── proposal/               # 人整理的信息
        ├── yyyymmdd_proposal.md  # 人整理的信息
```

这种目录结构的好处是：

- **清晰分层**：`stages` 目录存放 Agent 自动生成的文档，`proposal` 目录存放人工整理的原始需求
- **可追溯**：`workflow_events.jsonl` 记录了整个工作流的执行过程，便于问题排查
- **可复用**：每个需求的 P1–P4 文档都是独立的、可复用的资产
- **可并行**：每个需求独立的 Spec 目录让 Agent 可以并行为多个需求推进 P1–P3 阶段，数据研发同学只需在关键节点（P1 确认、P3 CR）介入，实现多需求并行处理

Skill 工作流

整个Skill本质上是**分阶段的 Spec 的编程范式**：

痛点	对应阶段	解决方式	Spec Coding 视角
😩 手工搜集 Prompt 信息累	**P1 + P2**：Agent 自动提取维度/指标 + 自动搜表查结构	人只需要提供原始需求，Agent 负责"翻译+找表"	**Spec = 需求澄清文档 + 资源关联文档**：在代码生成前，先把"做什么、用什么"确定
💥 复杂 SQL 生成质量差	**P3**：产出结构化伪代码 + 规范化 CTE，把复杂度拆解为模型吃得消的粒度	不是让 Copilot 猜，是给它一份参考蓝图	**Spec = 模型设计文档**：DDL + 伪代码就是"可 review 的规格书"，LLM 只需翻译 Spec → Code
🔄 多轮对话退化	**P4**：一份独立、自包含的结构化 Prompt，每次新对话粘贴即可	不需要依赖历史上下文，彻底摆脱"熵增"	**Spec = 自包含的代码生成指令**：不依赖对话历史，一份 Spec 开一个新对话
📋 评审后缺标准方案	**P1-P4 全套文档**：每个需求产出一套完整的需求方案资产	需求方和数据方的"共同锚点"，消除隐性理解偏差	**Spec = 研发共识锚点**：4 份文档生成后，任何人回头看都能完整还原决策链

一言以蔽之：**普通的 AI 辅助编程是"你说需求，AI 写代码"；Spec Coding 是"你说需求 → AI 帮你写 Spec → 你 review Spec → AI 基于 Spec 写代码"**。这一步"review Spec"的引入，把不可控的"黑盒生成"变成了可审计的"分层交付"。

案例一：直播活跃用户统计（24h交付）

- 需求沟通

原需求描述如下，其中相关表和口径为需求沟通时确认。

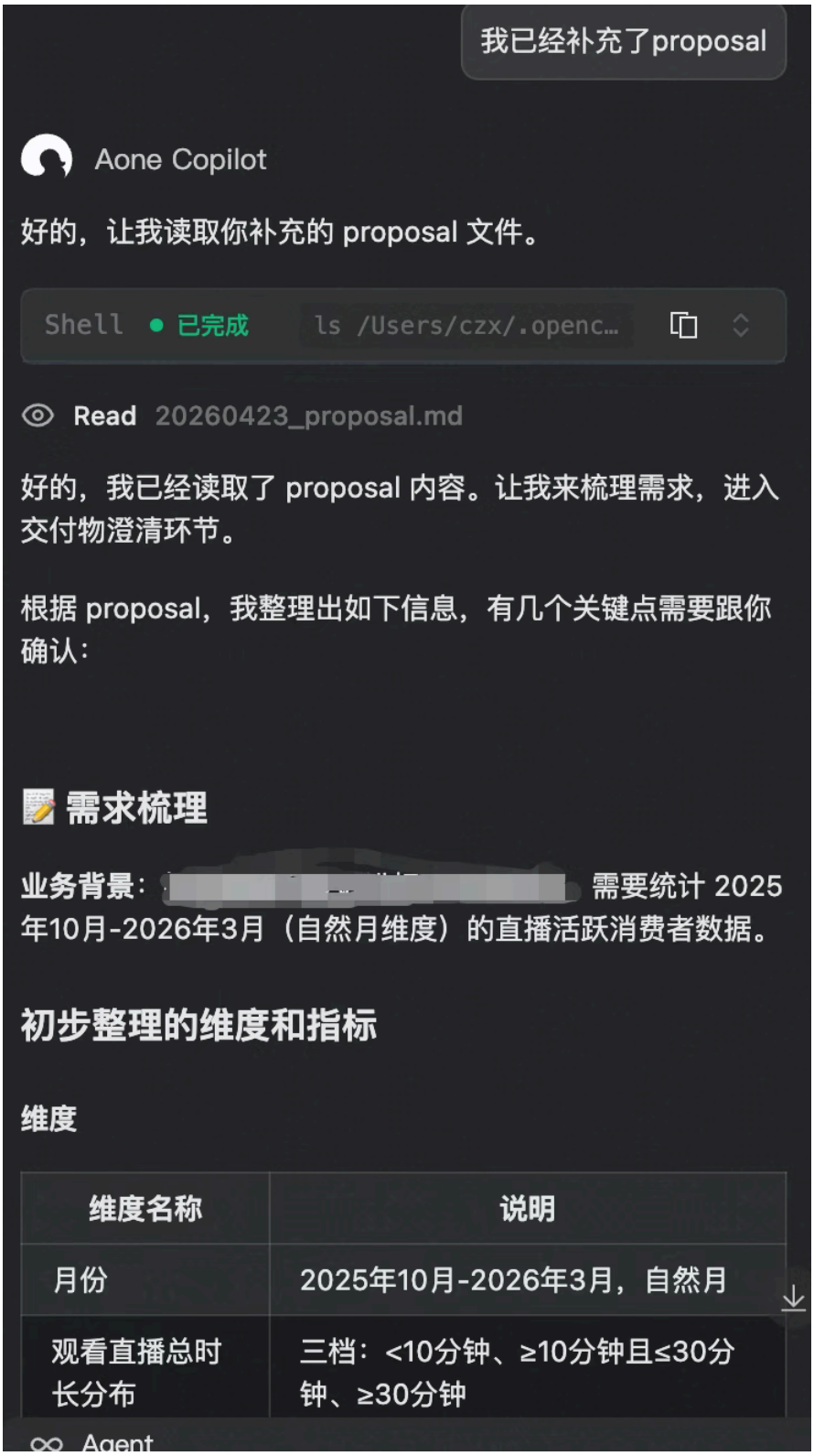
```
# 要求
时间周期：2025年10月-2026年3月，自然月的月维度数据
维度：月份，观看直播总时长分布（小于10分钟，大于等于10分钟且小于等于30分钟，大于等于
指标：活跃消费者数（去重uv）、活跃且确收消费者数（去重uv）
活跃消费者就是观看直播的消费者
# 相关表
## 活跃用户圈选
my_project.dim_user_tag
## 观看直播总时长
read_time
my_project.dws_vst_pv_1d
## 确收
my_project.dws_vst_trd_1d
```

- Skill使用过程

1. 新建spec目录



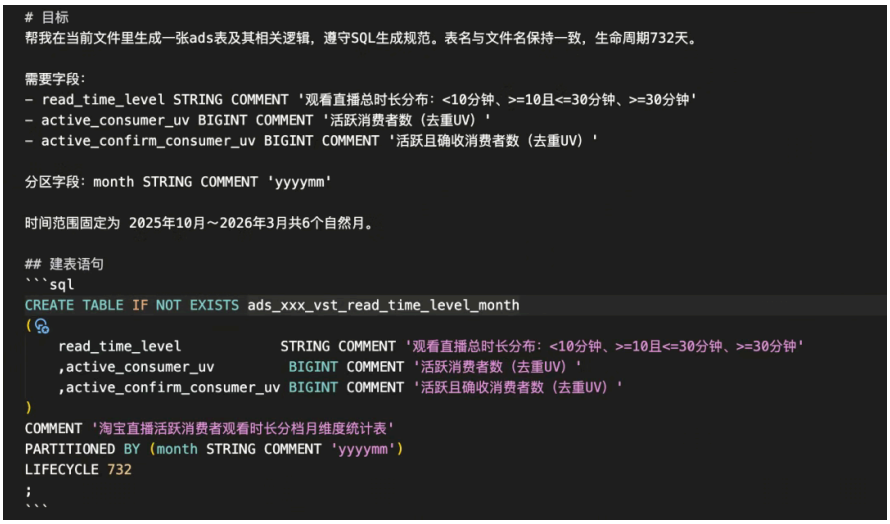
2. 需求澄清（将Aone需求描述复制到本地spec目录的proposal文件夹中）



3. 模型设计（AI自动推进，人工CR）



4. Copilot交互物（步骤3CR后AI自动推进）



5. 最终AI生成代码（0人工修改）



渐进式披露：Skill 的上下文管理设计

在深入介绍各个阶段之前，先讲一个容易被忽略但极其关键的设计维度——**渐进式披露（Progressive Disclosure）**——这也是 Skill 区别于传统"写一份 prompt 模板"的地方。

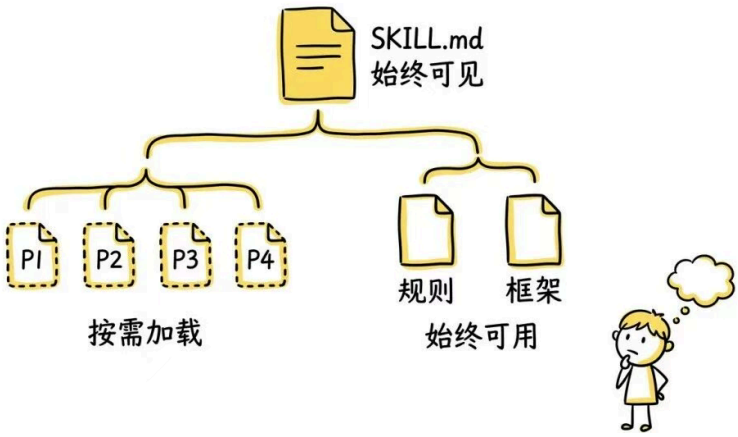
- 为什么渐进式披露对 Agent Skill 至关重要

大模型有一个"反直觉"的特性：**更多上下文不等于更好输出**。当你把所有规则、模板、示例、规范一股脑塞进 Prompt 时——

- 模型容易"迷失中间"（Lost in the Middle）：关键约束被淹没在样板信息中
- 注意力被稀释：模型不确定哪些是"必须遵守的"、哪些是"仅供参考的"
- 不同任务的规则互相干扰：查表规则和模型设计规则混在一起，模型容易张冠李戴

渐进式披露要解决的问题就是：**在正确的时间，给模型正确量级的信息**。

- Skill 中的渐进式披露设计




```
SKILL.md                          <- Agent 始终可见
  ├── references/
  │   ├── requirement_example/    <- 仅 P1 阶段按需加载
  │   ├── resource_example/       <- 仅 P2 阶段按需加载
  │   ├── model_design_example/   <- 仅 P3 阶段按需加载
  │   ├── copilot_input_example/  <- 仅 P4 阶段按需加载
  │   ├── etl_rules/              <- 全阶段共享（规范约束）
  │   └── workflow_explaination/  <- 框架层（状态管理）
```

主文件 SKILL.md 只有 4 阶段的路由定义和核心规则，约 200 行。每个阶段的具体执行模板、示例、规范文档在 `references/` 的 7 组文件中，Agent 在执行到对应阶段时才按需加载。

阶段1: 交付物定义

Skill 的第一项核心能力是从自然语言需求中提取维度列表和指标列表。举个例子：

原始需求：「帮我看下 20251009 到 20251111 各玩法类型的红包效果，包括曝光、领取、核销和成交」

Agent 自动提取：

类别	内容
维度	玩法类型
指标	红包曝光 PV、红包曝光 UV、领取金额、领取 UV、核销 UV、核销金额、红包引导成交金额
时间范围	2025-10-09 至 2025-11-11

提取完成后，agent 会逐项跟你确认。这个过程遵循严格的**澄清原则**：

- **一次一个问题**：不同时问多个问题，避免信息混乱
- **多选优先**：能给选项就给选项，减少你的输入负担
- **追问模糊点**：关键信息不明确时追问，不猜测
- **不修改原始字段名**：完全保持用户输入的字段名称
- **有表名就填、无表名写"暂无"**：绝不自行推断未明确给出的内容

阶段2：表、口径搜索 + 校验

阶段 2 的核心任务是**把 P1 里澄清的维度和指标，映射到具体的表和口径上**。这相当于给 Agent 装上一份"地图"——没有地图，Agent 只能在黑暗里猜；有了地图，它才知道每一步该走哪条路。

想要让模型生成正确的SQL代码，用户需要给出明确的来源表和指标口径。这不是阶段 2 才做的事——它应该在阶段 1 的交付物澄清中就搞定。P1 定义的每一个指标，最好都应该附带：

- **指标口径**："用什么字段？是否需要额外加工？加工粒度时什么"
- **来源表**："这个指标从哪张表取？"
- **过滤条件**："有什么限制条件？"

但是，对数据研发同学来说，根据需求中的交付物定义，找到对应的指标和表也是需要耗时耗力的。我们可以从使用频率将指标和口径分为常用和不常用两类。

类型	特点	处理方式
常用的	团队反复使用的指标/维度，口径稳定，来源表和过滤条件明确（如"直播间曝光 PV""红包核销 UV""确收成交金额"）	维护在**语义层知识库**中，Agent 自动检索匹配，秒级返回
不常用/临时的	某个具体需求的特殊口径，以前没做过，或一次性取数（如"取跟着事件类型 1019 后的第一条 1021/1023/1026"）	**人工探查**：自己写 SQL 片段验证、查 DDL 确认字段是否存在，然后把结果告诉 Agent

直播数据团队已经建设了语义层知识库，常用口径可以在本地知识库中自动搜索。Agent 在搜索完成后会自动查询表结构、确认字段是否满足，最后让用户确认即可。

但对于不常用、临时的口径或者新引入的表，**语义层帮不了忙**。这时候需要有经验的数据研发同学手动探查——跑一段验证 SQL、确认字段类型、检查分区是否存在——然后把探查结果告诉 Agent，由 Agent 整理到 P2 资源文档中。这是人工经验发挥价值的地方，也是 LLM 目前无法替代的环节。

关键优势：多需求并行推进

传统模式下，每个需求的前置准备工作（查表、确认口径、整理方案）都需要数据研发同学亲自完成，只能串行处理。而通过 Skill，多个需求可以同时启动 P1–P3 阶段的自动化流程：

- **需求 A**：Agent 正在执行 P2 搜索表结构
- **需求 B**：Agent 正在执行 P1 需求澄清
- **需求 C**：等待 P3 模型设计的 CR 确认

数据研发同学只需在关键决策点介入（P1 确认、P3 CR），其他时间 Agent 可以并行为多个需求推进，**将原本的"串行排期"变为"并行推进"**，大幅提升多需求场景下的交付效率。

阶段3：模型设计

阶段 3 是整个 Skill 流程中**最重要、也最需要**人工介入的环节。这里有一个必须想明白的因果关系链：

```
P3 模型设计质量 → P4 交互物质量 → Copilot 生码质量
```

如果 模型设计 的 DDL 分区策略写错了，P4 就会带着错误的分区策略传给 Copilot；如果 模型设计 的伪代码 JOIN 逻辑有问题，P4 就会让 Copilot "翻译"一段有问题的逻辑；如果 模型设计 的字段映射漏了一个关键字段，P4 就不会告诉 Copilot 这个字段的存在。

- 哪些规范要写在 Rule 里

通用规则可以沉淀。直播业务的名词规范、指标命名规范、表命名规范、模型设计原则等，这些是长期积累的知识，应该写入 Skill 的 Rule 层，让每次调用都能自动遵循。

业务特定规则需要人工决策。比如"这个需求应该用宽表还是星型模型"、"数据应该按天分区还是按小时分区"、"这个指标应该用累加值还是快照值"——这些决策依赖业务场景和数据特性，无法完全自动化，需要数据研发同学的专业判断。

- 为什么一定要用户 CR

直播数仓的规范很复杂，即使使用了渐进式披露，大模型也难以100%遵循。直播数仓的表命名规范、指标命名规范、业务域黑话、名词解释、代码规范等相当复杂，即使这些规则放在参考文件里，通过渐进式披露让 LLM 自主使用，也很难完全遵守。

规范再完善，也无法替代人对具体问题的理解与判断。对业务逻辑的理解、对技术边界的把握、对潜在问题的预判——这些能力无法通过几个 Markdown 文件传递给 AI。Agent 可以根据文档规范自动完成 90% 的设计工作——加工步骤、流程图、DDL、伪代码等。**人工 CR 的作用是校验规范的遵循情况，以及对模型给出的技术方案，做最后的确认。**

▌阶段4：Copilot 交互物 —— 把"设计方案"变成 Copilot 的"施工图纸"

如果说阶段 3 是"设计蓝图"，阶段 4 就是"施工图纸"——它负责把 P3 确认过的模型设计，翻译成一份 Copilot 可以直接执行的结构化 Prompt。

P4 的质量上限由 P3 决定。 P3 的 DDL 对了，P4 才可能对；P3 的伪代码逻辑正确，P4 才可能让 Copilot 生成正确的 SQL。所以上一节强调的 P3 人工 CR，本质上就是在为 P4 的质量"上保险"。

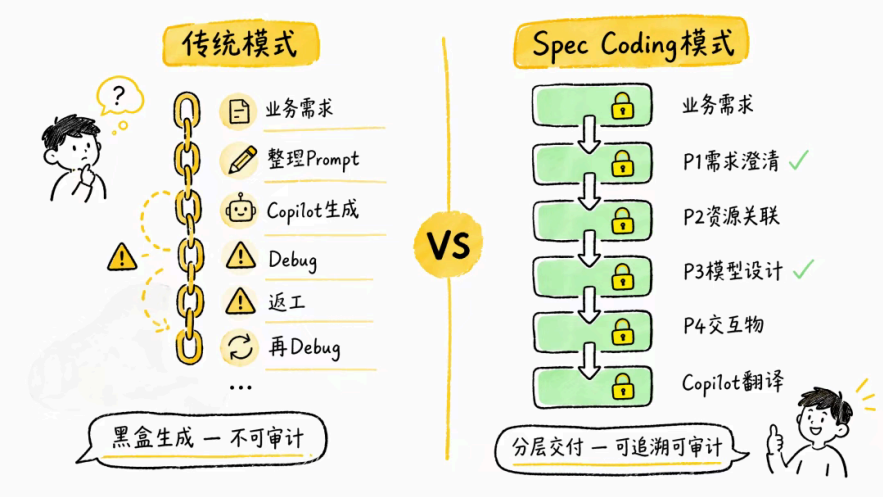
第一，P4 解决的是上下文问题，不是能力问题。 即使 P4 写得再完美，Copilot 生成的 SQL 仍需人工 CR。Spec Coding 让 AI 的输出"更可被审计"，但"审计"这件事仍然需要人来做。P4 本身就是渐进式披露思想的一个体现——Copilot 不需要知道 P1 的澄清过程、P2 的字段映射讨论、P3 的 DDL 演变历史，它只需要看到"最终结论"。

▌Spec Coding 在 SQL 生码的应用

- 传统模式对比

到这里，你可能已经注意到一个模式：**每一阶段产出的都是文档（Spec），而不是代码。**

这是有意为之。让我们对比两种范式：



传统模式：每一步都没有"可确认的中间产物"。debug 是在代码层修的，prompt 是脑子里组织的，模型为什么写错了也不知道。两周后回头看，谁也说不清当初为什么这么写。

Spec Coding 模式：需求 → Spec → 代码

每一步都有一个**可 review、可确认、可追溯的文档锚点**。好处非常具体：

对抗痛点 1（手工搜集 Prompt 累）→ 自动化：P4 包含的所有信息——需求背景、维度/指标清单、来源表+字段映射、DDL、伪代码——全部由 Agent 在 P1–P3 阶段自动生成，你不需要手动整理一个字。

对抗痛点 2（复杂 SQL 效果差）→ 把设计复杂度消化在 P3：Copilot 拿到的 P4 里，CTE 命名、JOIN 逻辑、聚合方式、过滤条件都已在 P3 确认过。Copilot 不需要"设计"，只需要"翻译"——把伪代码翻译成规范格式的真实 SQL。

对抗痛点 3（多轮对话退化）→ Spec 的"无状态"优势：P4 是独立的、自包含的 Prompt——不引用历史、不依赖上下文。新对话只需要引用最终的 Spec 产物（P4），不需要携带整段对话历史。本质上是把 Copilot 从"有状态服务"降级为"无状态服务"。

对抗痛点 4（评审后缺少标准方案）→ 可确认的中间产物：传统的 Code Review 是在代码写完了才做的。但 Spec Coding 把 Review 前置了

- 交付物口径明确了吗？（P1 review）
- 相关表对了吗？字段映射对了吗？（P2 review）
- 数据模型设计对吗？（P3 review）

在代码还没生成之前，大量可能的问题就已经被拦截了。

对抗痛点 5（多需求串行排期效率低）→ Agent 并行推进，人按优先级介入：每个需求独立的 Spec 目录让 Agent 可以并行为多个需求执行 P1–P3 的准备工作。数据研发同学不需要"守着"一个需求从头跟到尾，而是可以同时推进多个需求——在 Agent 执行 A 需求的 P2

搜索时，去确认 B 需求的 P1 澄清结果，再去 CR C 需求的 P3 模型设计。**把"依赖人的连续投入"的串行模式，转变为"人按优先级介入决策"的并行模式。**

用一句话总结：**Spec Coding 不是让 AI 更聪明，而是让 AI 的工作"更可被人类审计"**。skill 中的每一步强制确认（阶段 1 和阶段 3 必须用户确认才能继续），本质上就是在执行"human-in-the-loop 的 Spec Review"。

- Spec Coding 在多需求场景下的价值

当数据研发同学同时承接多个需求时，Spec Coding 的价值尤为突出。传统模式下，每个需求的前置准备工作（查表、确认口径、制定方案）都需要人的连续投入，只能串行完成。而 Spec Coding 模式下：

- **Agent 并行推进**：多个需求的 P1-P3 阶段可以同时进行，Agent 自动完成需求澄清、表结构查询、模型设计草案
- **人按优先级介入**：数据研发同学不需要"守着"一个需求，而是可以灵活地在多个需求之间切换——在 A 需求的 P3 等待 CR 时，去确认 B 需求的 P1，去补充 C 需求的 P2 表信息
- **异步协作模式**：每个需求独立的 Spec 目录成为异步协作的载体，人和 Agent 可以在不同时间点介入，无需同步等待

这种模式将**“依赖人的连续投入的串行排期”转变为“Agent自动执行，人按优先级介入决策的并行推进”**，在多需求场景下大幅提升交付效率。

Spec Coding 的边界

Spec Coding 做的是"翻译层"的事，不是"决策层"的事。 让我们明确它的能力边界：

问题域	Spec Coding 能做什么	Spec Coding 不能做什么
上下文管理	✅ 把需求确认为自包含的 P4，切断对长对话历史的依赖	—
过程追溯	✅ P1-P4 文档链让每一步决策都有证据	—
Review 前移	✅ 逻辑错误在 P3 伪代码阶段就能拦截	—
数据建模	—	❌ 表关联设计、DDL 定义、CTE 拆解必须由人来做
指标口径判断	—	❌ 交付物具体的指标、维度和口径，依赖人来沟通确认
SQL 正确性保证	—	❌ P4 再好，模型生成 SQL 仍有逻辑错误风险，必须人工 CR
复杂业务逻辑	—	❌ 复杂业务逻辑的翻译和实现仍依赖人的经验

一句话：Spec Coding 降低了"沟通成本"和"上下文管理成本"，但没有降低"需求本身的复杂度，对于依赖专家经验和判断的步骤，还是存在**人工成本**。

这也是为什么 Skill 的流程里，P1 和 P3 是强制确认节点——不是形式主义，而是因为**数据建模和指标定义这两个决策环节，大模型无法替代人的专业判断**。Code Review 同样是不可跳过的，在数据领域，一个错误的 JOIN 可能导致整张表的数据偏差，这种风险只能靠人的经验来兜底。

所以 Spec Coding 的正确定位是：它不是"让 AI 自动完成数据开发"，而是"让 AI 辅助的每一步都留下可追溯、可审查的文档锚点"。真正的模型设计、指标判断、代码审查——这些压舱石仍然在人身上。



当前成果

痛点	解决程度	具体表现
手工搜集 Prompt 累	✅ 基本解决	P1+P2 自动化维度提取 + 自动搜表，人只需提供原始需求
复杂 SQL 效果差	✅ 大幅改善	P3 伪代码做蓝图，Copilot 从"设计"降级为"翻译"，准确率 50%→80%
多轮对话退化	✅ 工程化规避	P4 自包含 Prompt，每次新对话粘贴即用，不依赖历史
评审后缺标准方案	✅ 结构化解法	P1-P4 全套文档 = 需求方和数据方的"共同锚点"
多需求串行排期	✅ Agent自动化执行	Agent自动执行，人按优先级介入决策的并行推进

1. 文档清晰、口径定义准确的临时取数需求（如维表开发、增量表仿写），已经做到**24小时内交付**。业务方的"Aone需求"→ Skill 澄清 → Copilot 生码 → 验数上线，全链路已经跑通。
2. Copilot 生码质量提高，对话轮次降低，复杂需求手写比率下降。
3. 需求从串行变成并行，缩短排期时间，提升交付效率。

还在探索的

规范的模型设计：遵循研发规范，在需求理解模型设计的过程中更好的理解研发规范，辅助人进行复杂的数据模型设计，找到最优的解法。

资产搜索自动化：目前还是依赖语义层，如何自动化探索，甚至在未来能更近一步前置资产搜索，让 AI 帮助业务方快速找到资产，提升效率。

数据需求质量提升：数据需求本身的质量高低已经成为了需求交付的时效性的强卡点。接下来需要再往前一步，总结Aone需求范式，提高需求本身的质量（描述清晰，无二义性，给出明确口径），在需求提出和需求评审阶段解决AI生码相关的需求澄清问题，**期望达到部分需求评审通过后不排期直接交付**。



本文作者冷星，来自淘天集团–直播技术数据团队。我们是一支充满活力、在AI赛道上高速成长的技术先锋队，深耕AI与直播深度融合的黄金赛道——从AI数字人到多模态大模型，从沉浸式音视频体验到智能互动购物。直播技术数据团队不断探索AI+DATA的新范式，致力于打造 AI 原生的直播数据研发新范式，用AI重新定义直播电商的未来。

¤ 拓展阅读 ¤

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 2747

留言

写留言




还没有留言，来写首评